

Mapping Entity Relationship (ER) diagrams to a relational schema

Sistemas de Informação 1

Credits and Disclaimer

- This material/slides are adapted from:
 - SINF1 2024/25 slides produced by Telmo Matos
 - Books:
 - Database Systems - A Practical Approach to Design, Implementation, and Management, Thomas Connolly & Carolyn Begg; Pearson 2015; 6th Edition (Global Ed.); ISBN: 978-1-292-06118-4



ER to logical

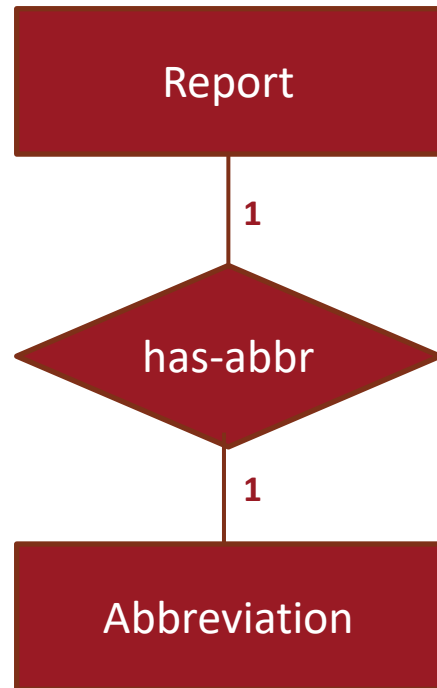
- Can produce **three types** of tables:
 - Table with the same information as the original entity (from the ER model) from which it was derived;
 - **Entity → table**
 - Table containing the foreign key of the relationship with the associated entity;
 - **1:N relation → FK**
 - Derived table from the relationship, containing the foreign keys of all entities involved in the relationship.
 - **N:N relation → associative table**

ER to logical

- The following rules apply to the handling of null values in SQL in these transformations:
 - Nulls **are allowed** in a table for foreign keys of associated (referenced) optional entities;
 - Nulls **are not allowed** in a SQL table for foreign keys of associated (referenced) mandatory entities;
 - Nulls **are not allowed** for any key in a table derived from a many-to-many relationship, because only complete row entries are meaningful in the table.

ER to logical

- Every report has one abbreviation, and every abbreviation represents exactly one report.



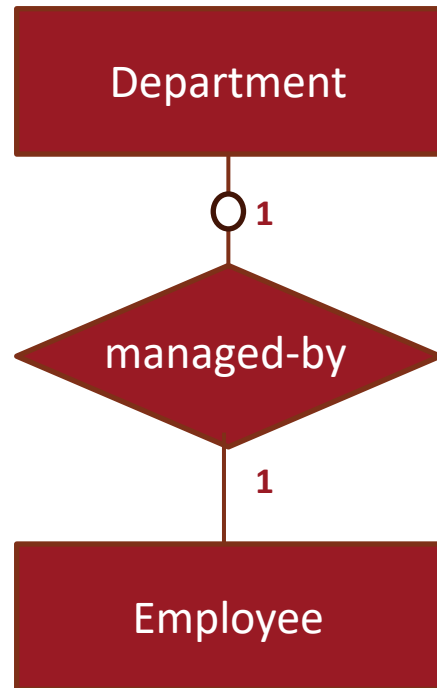
One-to-one, both entities mandatory

```
create table report (  
    report_num integer,  
    report_name varchar(256),  
    primary key (report_num)  
);
```

```
create table abbreviation (  
    abbr_num char(6),  
    report_num integer not null unique,  
    primary key (abbr_num),  
    foreign key (report_num) references report  
        on delete cascade on update cascade  
);
```

ER to logical

- Every department must have a manager, but as employee can be a manager of at most a department.

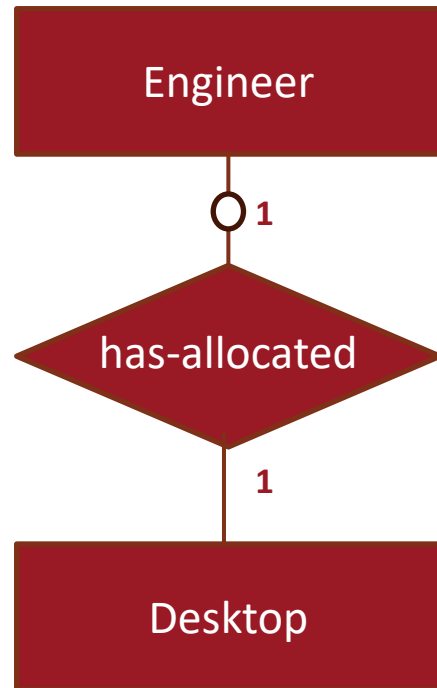


One-to-one, one entity optional, one mandatory

```
create table department (  
    dept_num integer,  
    dept_name char(20),  
    mgr_id char(10) not null unique,  
    primary key (dept_num),  
    foreign key (mgr_id) references employee  
        on delete set default on update cascade  
);  
create table employee (  
    emp_id char(10) default 'UNKNOWN',  
    emp_name char(20),  
    primary key (emp_id)  
);
```

ER to logical

- Some desktop computers are allocated to engineers, but not necessarily to all engineers.

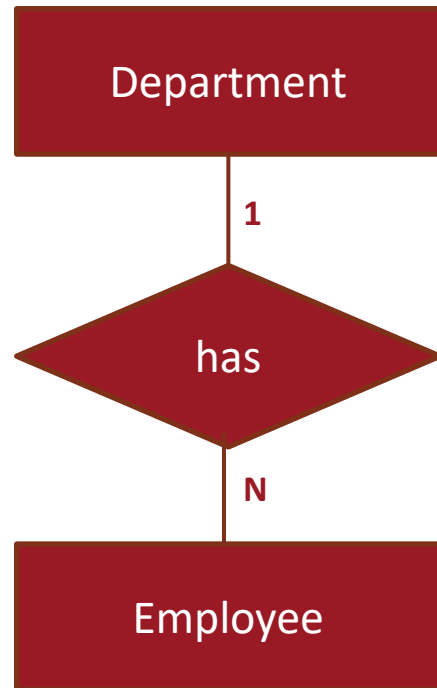


One-to-one, both entities optional

```
create table engineer (  
    emp_id char(10),  
    desktop_num integer,  
    primary key (emp_id)  
);  
  
create table desktop (  
    desktop_num integer,  
    emp_id char(10),  
    primary key (desktop_num),  
    foreign key (emp_id) references engineer  
        on delete set null on update cascade  
);
```

ER to logical

- Every employee works in exactly one department, and each department has at least one employee.



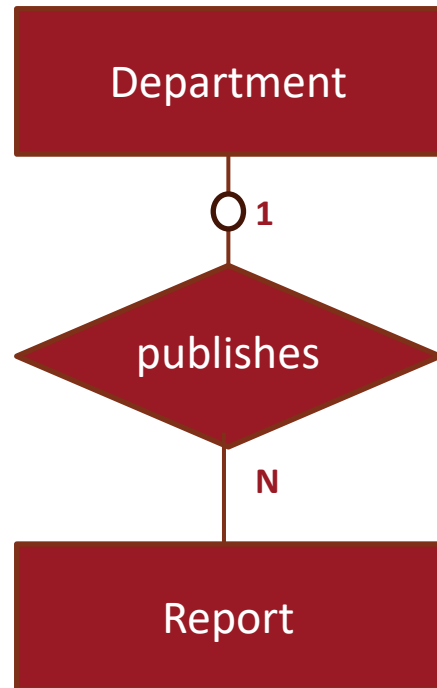
One-to-many, both entities mandatory

```
create table department (  
    dept_num integer default 0,  
    dept_name char(20),  
    primary key (dept_num)  
);
```

```
create table employee (  
    emp_id char(10),  
    emp_name char(20),  
    dept_num integer not null,  
    primary key (emp_id),  
    foreign key (dept_num) references department  
        on delete set default on update cascade  
);
```


ER to logical

- Every department must have a manager, but as employee can be a manager of at most a department.



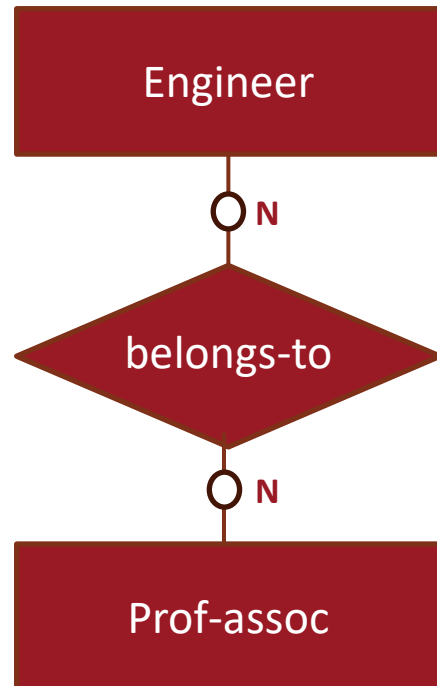
One-to-many, one entity optional, one mandatory

```
create table department (  
    dept_num integer,  
    dept_name char(20),  
    primary key (dept_num)  
);
```

```
create table report (  
    report_num integer,  
    dept_num integer,  
    primary key (report_num),  
    foreign key (dept_num) references department  
        on delete set null on update cascade  
);
```

ER to logical

- Every professional association could have none, one, or many engineer members. Each engineer could be a member of none, one, or many professional associations.



Many-to-many, both entities optional

```
create table engineer (emp_id char(10), primary key (emp_id));
```

```
create table prof_assoc (
    a_name varchar(256), primary key (a_name));
```

```
create table belongs_to (
    emp_id char(10),
    a_name varchar(256),
    primary key (emp_id, a_name),
    foreign key (emp_id) references engineer
        on delete cascade on update cascade,
    foreign key (a_name) references prof_assoc
        on delete cascade on update cascade
```

```
);
```

Cheat Sheet — ER → Relational Model (SQL)

■ Core Idea

- Transform:
 - **Entities** → **Tables**
 - **Relationships** → **Foreign Keys** or **new tables**

Cheat Sheet — ER → Relational Model (SQL)

■ Entities

- Each entity becomes a table
- Primary Key identifies each record

Cheat Sheet — ER → Relational Model (SQL)

■ One-to-one (1:1) Relationships

- Place **FK** in one table
- Use **UNIQUE**
- Choose side with fewer **NULLs**

Cheat Sheet — ER → Relational Model (SQL)

■ One-to-many (1:N) Relationships

- **FK** goes on N-side
- Mandatory → **NOT NULL**
- Optional → **NULL** allowed

Cheat Sheet — ER → Relational Model (SQL)

- Many-to-many (N:N) Relationships
 - Create a junction table
 - **PK** = combination of both **FK**

Cheat Sheet — ER → Relational Model (SQL)

- **NULL** rules
 - Optional **FK** → **NULL** allowed
 - Mandatory **FK** → **NOT NULL**
 - N:N → no **NULL**s

Cheat Sheet — ER → Relational Model (SQL)

■ Integrity Constrains

- Use FOREIGN KEY
- ON DELETE: SET NULL / CASCADE / RESTRICT

Cheat Sheet — ER → Relational Model (SQL)

■ Common Mistakes

- Duplicating **FKs**
- Missing **UNIQUE** in 1:1
- **NULL** in **PK**
- Wrong names